

Switching Circuits and Logic Design (CS21202)

Module 7 **Registers and Counters**

Prof. Indranil Sengupta
Department of Computer Science and Engineering
IIT Kharagpur

1

Design of Registers

2

Introduction

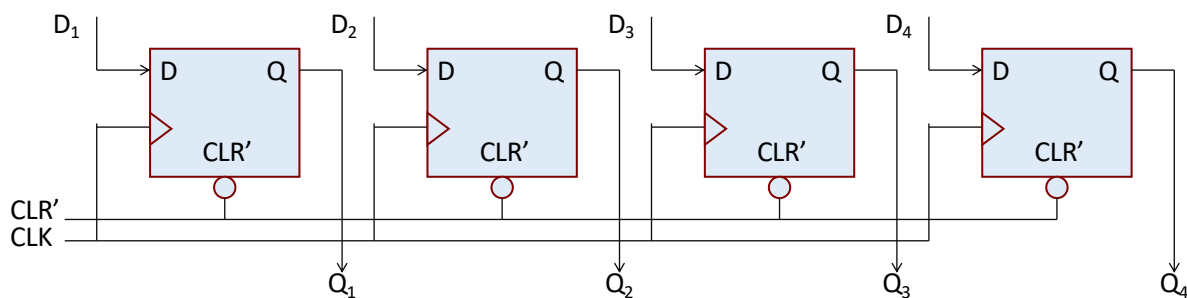
- A **register** consists of a group of flip-flops with a common clock input, used for storing binary data.
- Depending on the configuration, there can be several different variations:
 - a) Parallel-in parallel-out (PIPO)
 - b) Serial-in serial-out (SISO)
 - c) Parallel-in serial-out (PISO)
 - d) Serial-in parallel-out (SIPO)
- If the register supports serial-in and serial-out modes, it is also called a **shift register**.

Switching Circuits and Logic Design (CS20202)

3

3

Basic PIPO Register



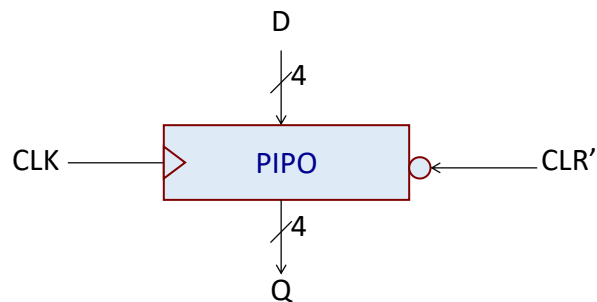
When the active clock edge arrives, input word $D_1D_2D_3D_4$ gets stored in the register, and available on the outputs $Q_1Q_2Q_3Q_4$.

Switching Circuits and Logic Design (CS20202)

4

4

Symbolic Representation (using vector notation)



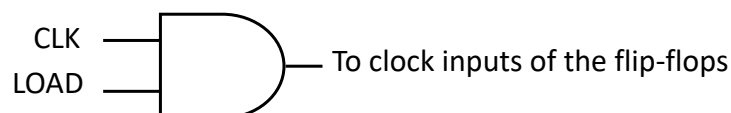
Switching Circuits and Logic Design (CS20202)

5

5

Addition of LOAD signal

- In practice, the clock is coming continuously, and there is a separate signal LOAD that specifies when the register is to be loaded with new data.
- Two possible solutions:
 - a) Use a gated clock:
 - Not a good solution, as gating the clock with another signal can cause timing problems.



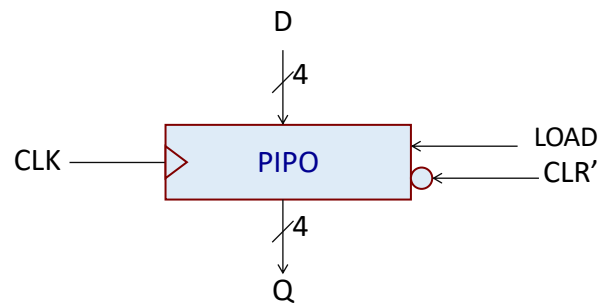
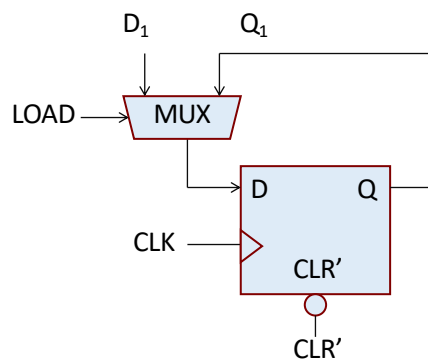
Switching Circuits and Logic Design (CS20202)

6

6

b) Separate out CLK and LOAD using a multiplexer circuit.

- Better and recommended solution.
- Each flip-flop in the register gets replaced by:



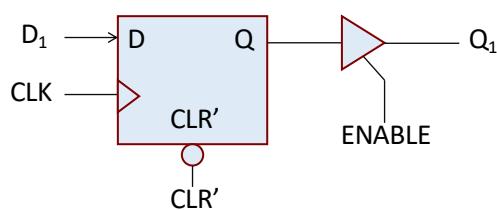
Switching Circuits and Logic Design (CS20202)

7

7

Addition of ENABLE input

- Many registers have an **ENABLE** input that can be used to force the output lines into high impedance state, if desired.
 - We need a tri-state buffer with every flip-flop output.



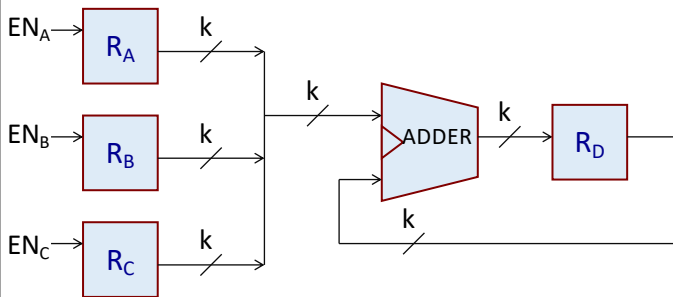
Switching Circuits and Logic Design (CS20202)

8

8

- Why is the **ENABLE** input required?

- To resolve bus conflicts in bus-based architectures.
- An example scenario:



- R_A , R_B , R_C , R_D are k-bit registers.
- Exactly one of the three enable inputs must be activated at any given time.
- CLK is fed in parallel to all the registers.
- If $EN_A=1$, $R_D = R_D + R_A$
- If $EN_B=1$, $R_D = R_D + R_B$
- If $EN_C=1$, $R_D = R_D + R_C$

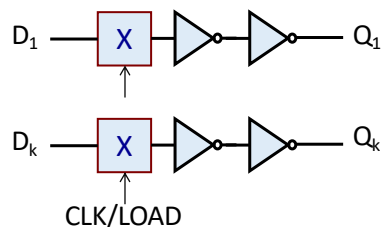
Switching Circuits and Logic Design (CS20202)

9

9

PIPO Register using CMOS Dynamic Logic

- In **dynamic storage cell**, information is stored not in flip-flops, but as charge stored on tiny capacitors.
 - Charge tends to leak away with time; and therefore needs some refreshing mechanism.
- Basic register structure, where X is a CMOS transmission gate:
 -

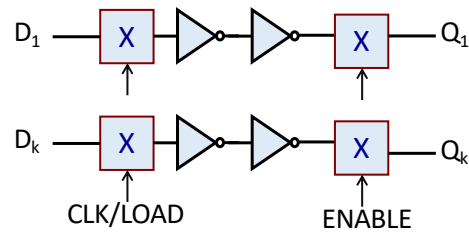


Switching Circuits and Logic Design (CS20202)

10

10

- For tri-state control, we can use another X-gate on the output side.

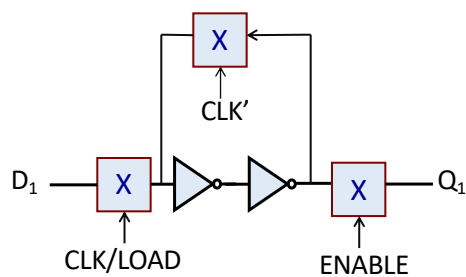


Switching Circuits and Logic Design (CS20202)

11

11

- For implementing refreshing logic, several solutions are possible.
- A simple solution is shown below for a single flip-flop stage:



Refresh operation is carried out in every clock period when $CLK = 0$.

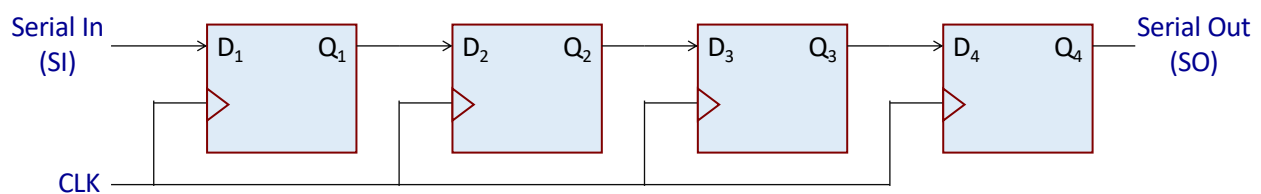
Switching Circuits and Logic Design (CS20202)

12

12

Shift Register

- A shift register is a register in which the binary data can be stored, and the data can be shifted to the left (or right) when a shift signal is applied.
 - New bit gets shifted in; bit shifted out typically get lost.
- Can be constructed simply by connecting D, SR or JK flip-flops in cascade.
- A 4-bit shift register is shown below:

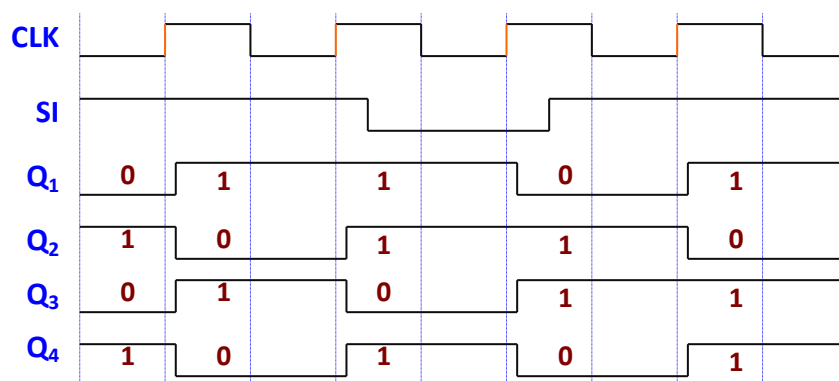


Switching Circuits and Logic Design (CS20202)

13

13

- Some shift registers have a shift enable signal **SHIFT**, which is used in the same way as the **LOAD** signal of a PIPO register.
 - Sample timing diagram is shown, assuming that the initial state is $Q_1Q_2Q_3Q_4 = 0101$.



Switching Circuits and Logic Design (CS20202)

14

14

Variations of Shift Registers

- Depending upon the way the various stages are connected, there can be various types of shift registers:
 - a) Ring counter
 - b) Twisted ring or Johnson counter
 - c) Bidirectional shift register
 - d) Universal shift register
 - e) Linear feedback shift register

Switching Circuits and Logic Design (CS20202)

15

15

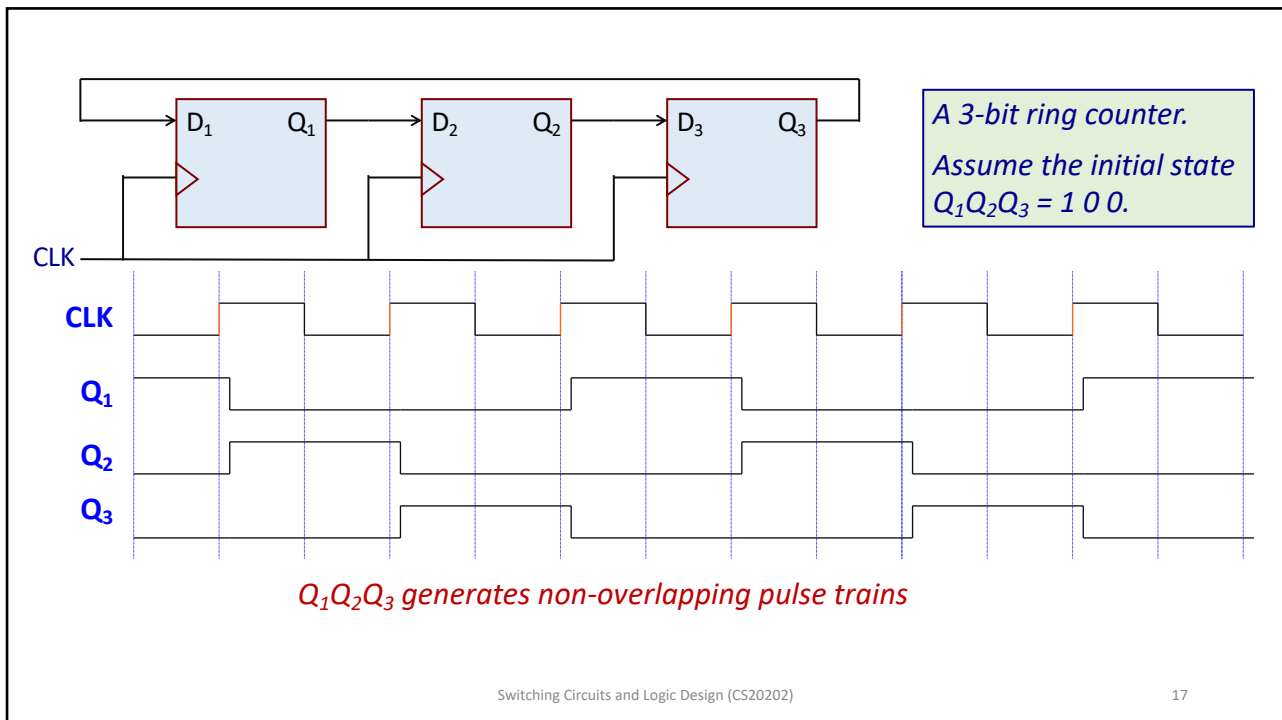
(a) Ring Counter

- This is obtained from a SISO shift register by connecting the Q output of the last flip-flop to the D input of the first flip-flop.
 - Typically, a ring counter is initialized with a single 1 and all remaining 0's.
 - This can generate multi-phase clock, or sequence of synchronizing pulses.
- For a k -bit ring counter, the contents of the register gets repeated after k clocks.
 - Modulo- k counter.

Switching Circuits and Logic Design (CS20202)

16

16



17

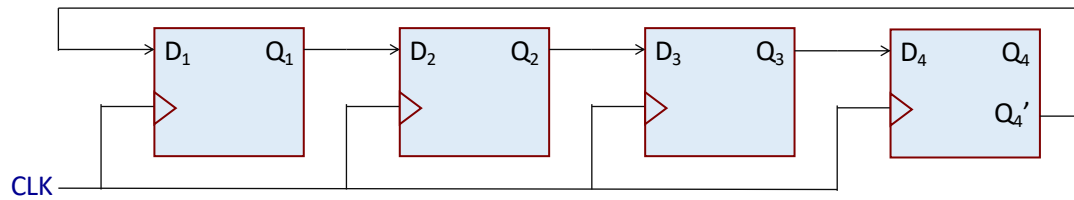
(b) Twisted Ring or Johnson Counter

- This is obtained from a SISO shift register by connecting the Q' output of the last flip-flop to the D input of the first flip-flop.
 - The Johnson counter can be initialized to the all-0 state.
 - The 0's and 1's are consecutive in the generated patterns (may be required for some applications).
- For a k -bit Johnson counter, the contents of the register gets repeated after $2k$ clocks.
 - Modulo- $2k$ counter.

Switching Circuits and Logic Design (CS20202)

18

18



Clocks		1	2	3	4	5	6	7	8	9	
Q_1	0	1	1	1	1	0	0	0	0	...	...
Q_2	0	0	1	1	1	1	0	0	0	...	...
Q_3	0	0	0	1	1	1	1	0	0	...	...
Q_4	0	0	0	0	1	1	1	1	0	...	...

A 4-bit Johnson counter.
Assume the initial state
 $Q_1Q_2Q_3Q_4 = 0000$.

Switching Circuits and Logic Design (CS20202)

19

19

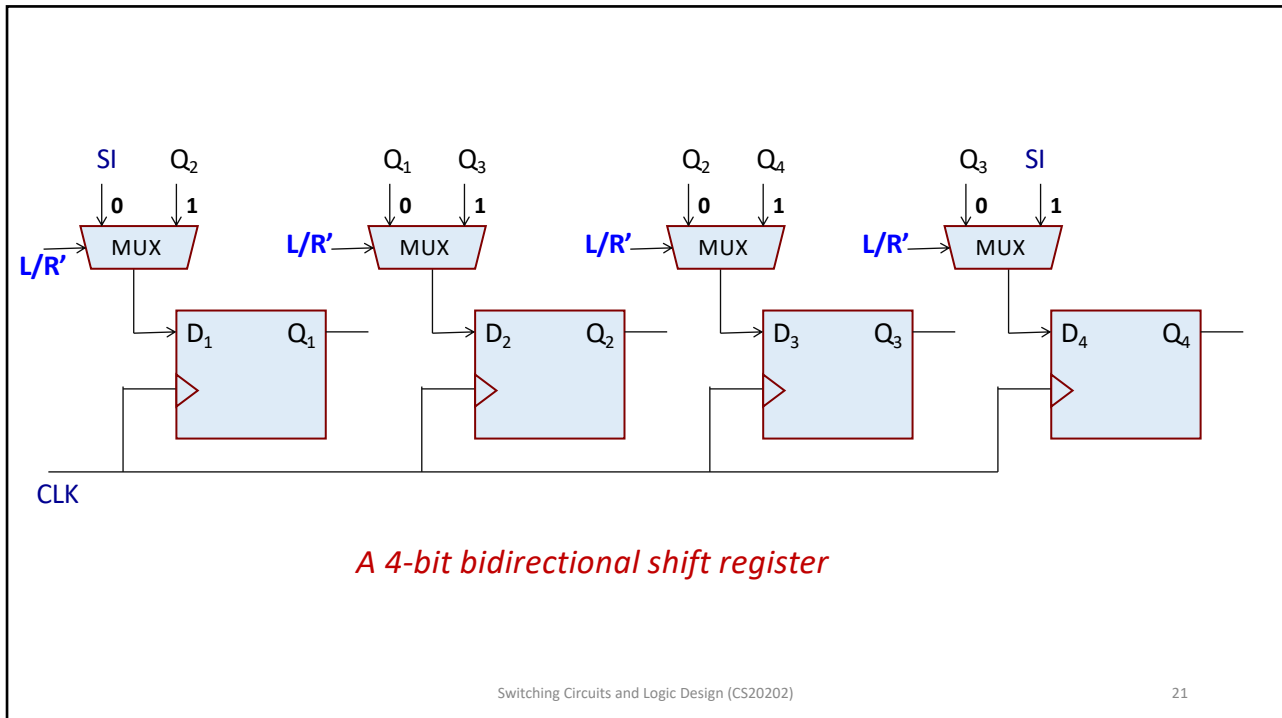
(c) Bidirectional Shift Register

- Based on a control input, the shift register works in either the shift-right or the shift-left modes.
- Multiplexers are used to feed the appropriate signals to the D inputs of the flip-flops.
 - A control input L/R' selects the multiplexer inputs.
 - Determines whether to shift left or to shift right.

Switching Circuits and Logic Design (CS20202)

20

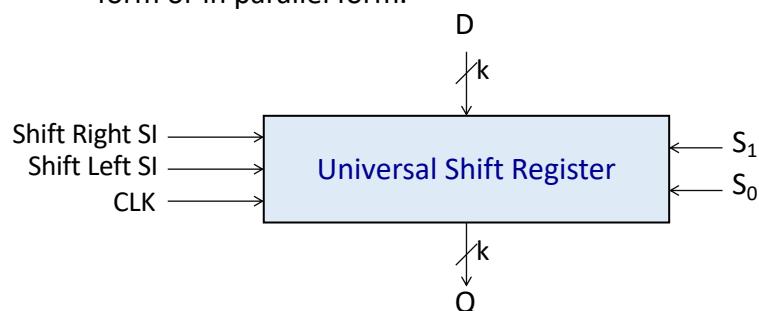
20



21

(d) Universal Shift Register

- A universal shift register is a bidirectional shift register, whose input can be either in serial form or in parallel form, and whose output can also be either in serial form or in parallel form.

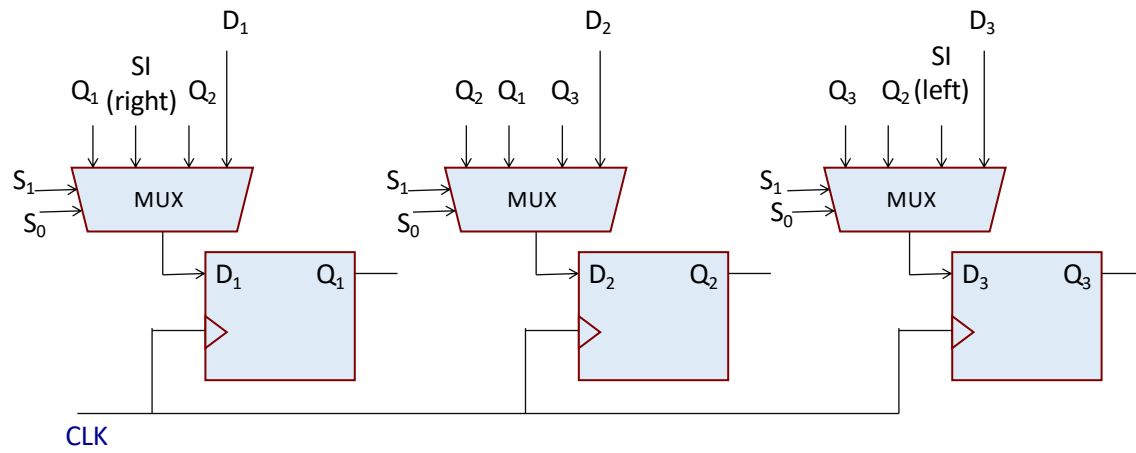


Control Inputs		Action
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

Switching Circuits and Logic Design (CS20202)

22

22



A 3-bit universal shift register

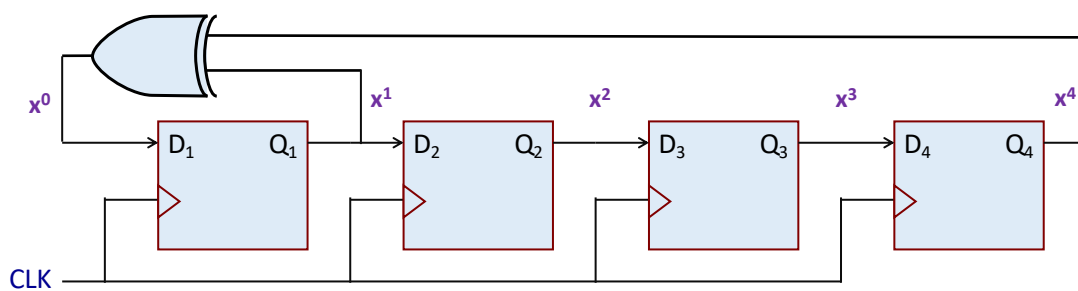
Switching Circuits and Logic Design (CS20202)

23

23

(e) Linear Feedback Shift Register (LFSR)

- It is basically a SISO right shift register, where the serial input is generated as a linear combination (exclusive-OR) of some of the flip-flop outputs.

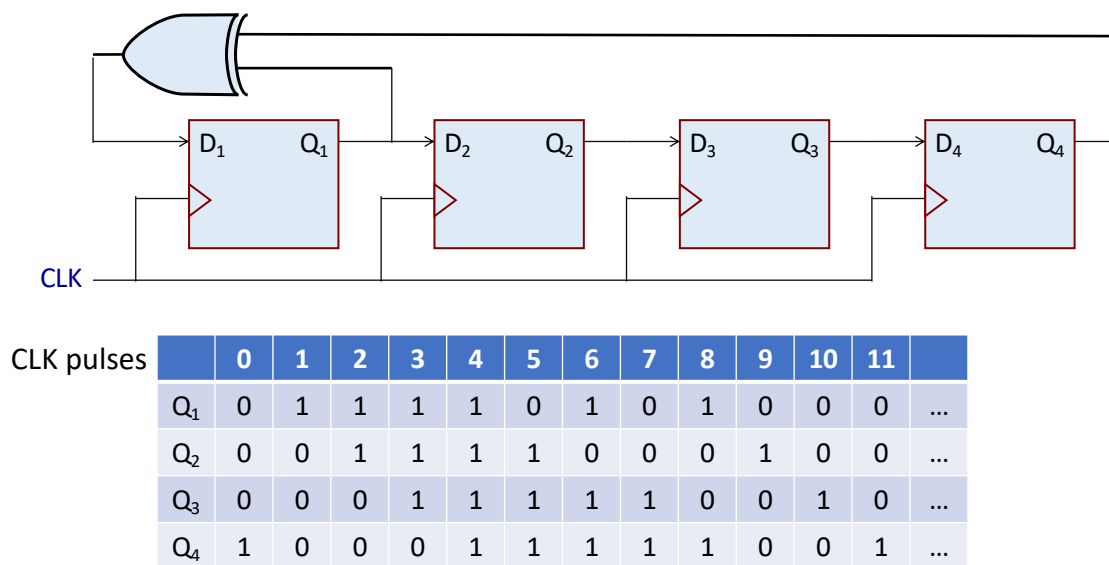


Characteristic polynomial :: $f(x) = x^4 + x^1 + 1$

Switching Circuits and Logic Design (CS20202)

24

24



Switching Circuits and Logic Design (CS20202)

25

25

- The LFSR is typically initialized to **1 0 0 ... 0**.
- If the taps are properly chosen, an n -bit LFSR can generate $2^n - 1$ distinct patterns without repeating.
- The LFSR generated patterns have very good randomness.
 - Useful in many applications.

Switching Circuits and Logic Design (CS20202)

26

26

LFSR – Some Terminologies

- If the sequence generated by an n -stage LFSR has period 2^n-1 , then it is called a *maximum-length sequence* or *m-sequence*.
 - Starting from any nonzero state, the LFSR goes through all 2^n-1 states before repeating.
 - Clearly, the all-zero pattern cannot be part of any m-sequence.
 - The patterns generated by a LFSR possess very good randomness properties (called pseudo-random sequence).
- The characteristic polynomial associated with maximum-length sequence is called a *primitive polynomial*.

Switching Circuits and Logic Design (CS20202)

27

27

Example Primitive Polynomials

3:	1 0	→ $x^3 + x + 1$
4:	1 0	→ $x^4 + x + 1$
5:	2 0	→ $x^5 + x^2 + 1$
6:	1 0	
7:	1 0	
8:	6 5 1 0	
16:	5 3 2 0	
32:	28 27 1 0	
64:	4 3 1 0	

Switching Circuits and Logic Design (CS20202)

28

28

Some Applications of Registers

1. Parallel-to-serial and serial-to-parallel conversion for data transmission

- Many device-to-device communication protocols use serial data transmission.
- Advantages: less number of wires, higher reliability, lower cost, etc.
- Data generators and receivers typically handle data in parallel.
- We require conversion, which can be done using shift registers.

Switching Circuits and Logic Design (CS20202)

29

29

2. Use in data path for performing computations

- For implementing complex systems, we typically break the design into two parts: (a) *Data Path* design, and (b) *Control Path* design.
- The Data Path contains the basic hardware units for performing calculation and storage (e.g. combinational circuits modules like adders, multipliers, comparators, multiplexers, etc. and storage elements like registers).
- The Control Path is a FSM that generates the control signals for the Data Path in proper sequence to carry out the required operation.
- Detailed design to be discussed later.

Switching Circuits and Logic Design (CS20202)

30

30

Design of Counters

Switching Circuits and Logic Design (CS20202)

31

31

Binary Counter

- A binary counter is a set of flip-flops whose states change in response to pulses applied at the input to the counter.
 - The combined state of the flip-flops at any time is the binary equivalent of the total number of pulses that have been applied.
- Counters are of two types:
 - a) Asynchronous counter
 - b) Synchronous counter

Switching Circuits and Logic Design (CS20202)

32

32

Asynchronous Counter (Ripple Counter)

- This type of counter is the easiest to design, and requires the least amount of hardware.
- The counter is called *asynchronous* because the flip-flops are not driven by the same clock signal, and hence do not change state simultaneously.
 - Constructed using T flip-flops.
- It is called *ripple counter* because when the counter, for example, goes from 1111 to 0000, it temporarily goes through a number of intermediate states: 1111 → 0111 → 0011 → 0001 → 0000.
 - Can lead to unwanted transitions if the outputs drive some other circuit.
 - Glitches are possible.

Switching Circuits and Logic Design (CS20202)

33

33

Synchronous Counter

- In this kind of counter, the same clock signal feeds all the flip-flops.
 - All the flip-flops change state simultaneously in synchronism with the clock pulse.
 - No glitches in the outputs.
 - Faster than asynchronous counters, but require more hardware for implementation.
- Can be designed using *Finite State Machine* (FSM) design methodology (to be discussed).

Switching Circuits and Logic Design (CS20202)

34

34

Design of 4-bit Ripple Counter (Up Counting)

- We make an observation:

- During counting, whenever a bit position changes from 1 to 0, the next higher bit position changes state.
- This feature can be directly implemented using a T flip-flop with *negative-edge triggered clock*.

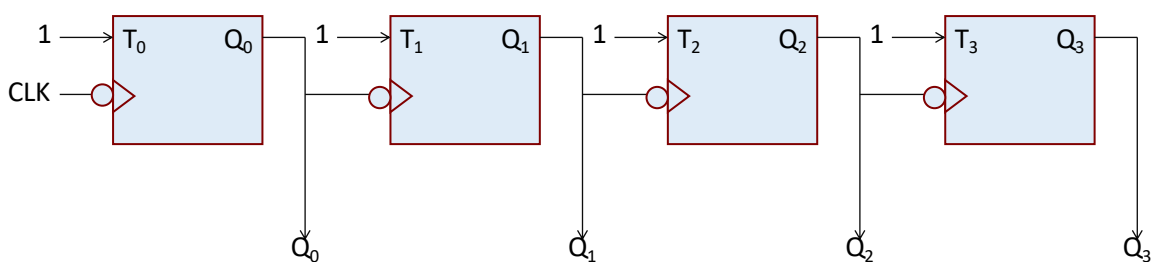
0: 0 0 0 0	8: 1 0 0 0
1: 0 0 0 1	9: 1 0 0 1
2: 0 0 1 0	10: 1 0 1 0
3: 0 0 1 1	11: 1 0 1 1
4: 0 1 0 0	12: 1 1 0 0
5: 0 1 0 1	13: 1 1 0 1
6: 0 1 1 0	14: 1 1 1 0
7: 0 1 1 1	15: 1 1 1 1

Switching Circuits and Logic Design (CS20202)

35

35

A 4-bit binary up counter



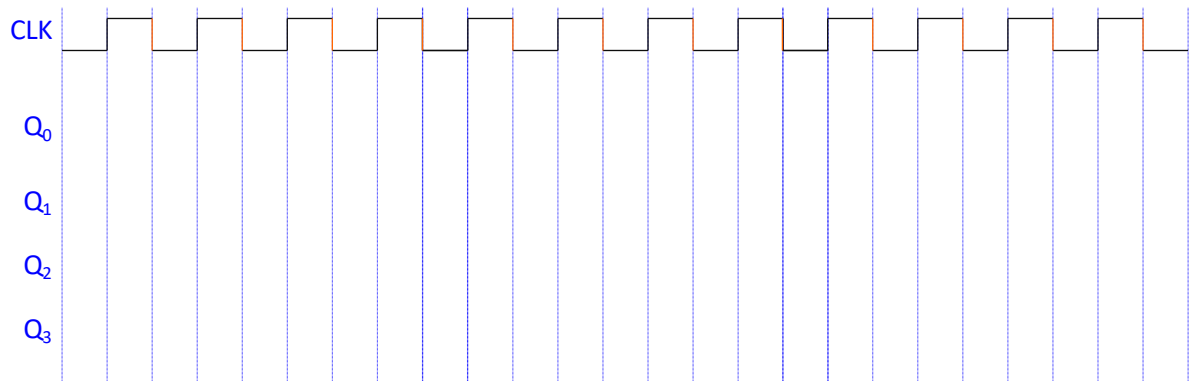
If we are designing an up counter using positive edge-triggered T flip-flops, we simply connect the Q' output of a stage to the clock input of the next stage.

Switching Circuits and Logic Design (CS20202)

36

36

Timing Diagram



Switching Circuits and Logic Design (CS20202)

37

37

- Some observations:
 - Suppose f denotes the frequency of the pulse train applied at CLK input.
 - Then, frequency of pulse train at $Q_0 = f/2$
 - Frequency of pulse train at $Q_1 = f/4$
 - Frequency of pulse train at $Q_2 = f/8$
 - Frequency of pulse train at $Q_3 = f/16$
- An n -bit binary counter is also called *modulo-2ⁿ* counter or *divide-by-2ⁿ* counter.

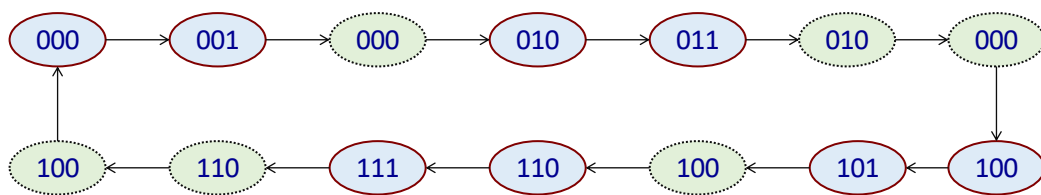
Switching Circuits and Logic Design (CS20202)

38

38

Transient States During Counting

- Consider a 3-bit ripple counter.
- The normal counting sequence is: 000, 001, 010, 011, 100, 101, 110, 111, 000, ...
- However, in practice, because of the delays of the T flip-flops, there are several transient states (shown dotted) that appear.



Switching Circuits and Logic Design (CS20202)

39

39

Design of 4-bit Ripple Counter (Down Counting)

- We make an observation:
 - During counting, whenever a bit position changes from 0 to 1, the next higher bit position changes state.
 - This feature can be directly implemented using a T flip-flop with positive-edge triggered clock.

15: 1 1 1 1	7: 0 1 1 1
14: 1 1 1 0	6: 0 1 1 0
13: 1 1 0 1	5: 0 1 0 1
12: 1 1 0 0	4: 0 1 0 0
11: 1 0 1 1	3: 0 0 1 1
10: 1 0 1 0	2: 0 0 1 0
9: 1 0 0 1	1: 0 0 0 1
8: 1 0 0 0	0: 0 0 0 0

Switching Circuits and Logic Design (CS20202)

40

40

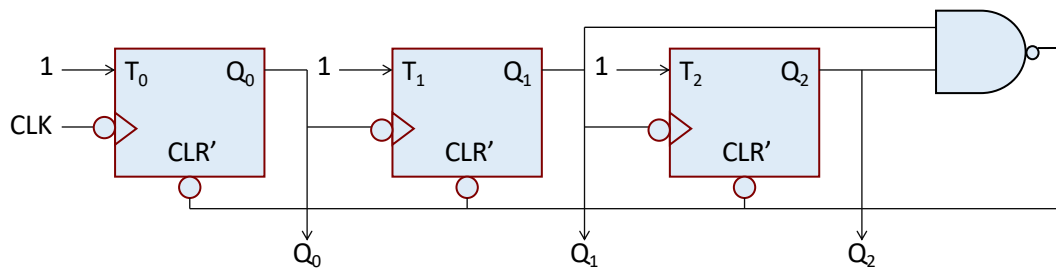
- If we use negative edge-triggered T flip-flops:
 - Connect Q' output of a stage to CLK input of next stage.
- If we use positive edge-triggered T flip flops:
 - Connect Q output of a stage to CLK input of next stage.

Design of Binary Ripple Counter of Arbitrary Modulus

- Suppose we want to design a *modulo-M* counter, for any arbitrary value of M .
 - Counter will count up from 0 to $M-1$, and then back to 0.
- Assume that the flip-flops have individual asynchronous CLR' inputs.
- Basic idea:
 - We first design a ripple counter with $\lceil \log_2 M \rceil$ stages.
 - Then we design a gating circuit, which takes inputs from the counter outputs, and generates a 0 whenever the count value reaches M .
 - Connect the output of the gating circuit to the CLR' inputs of the flip-flops.

Example: Design of a modulo-6 binary ripple counter

- Number of flip-flop stages required will be $\lceil \log_2 6 \rceil = 3$
- The desired count sequence will be: 000, 001, 010, 011, 100, 101, 000, ...
 - From state 101, instead of going to the state 110, we have to bring the state back to 000.

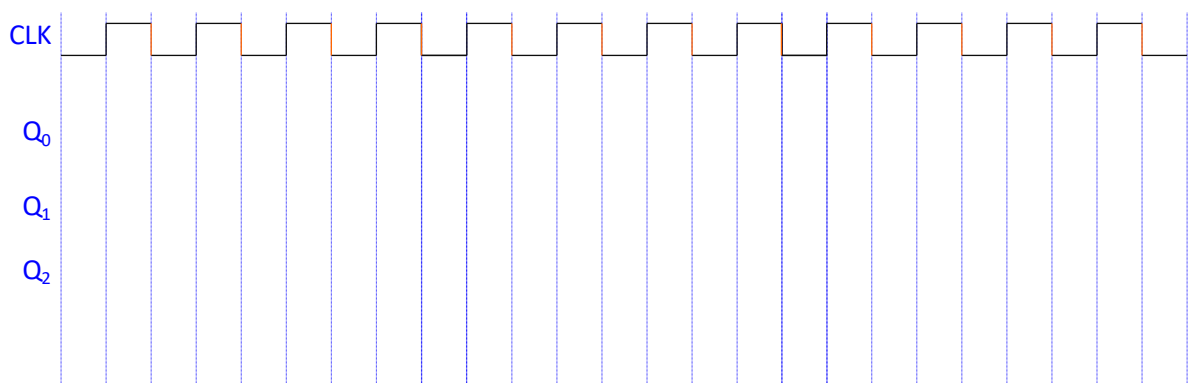


Switching Circuits and Logic Design (CS20202)

43

43

Timing Diagram :: Input frequency is getting divided by 6 at output Q_2

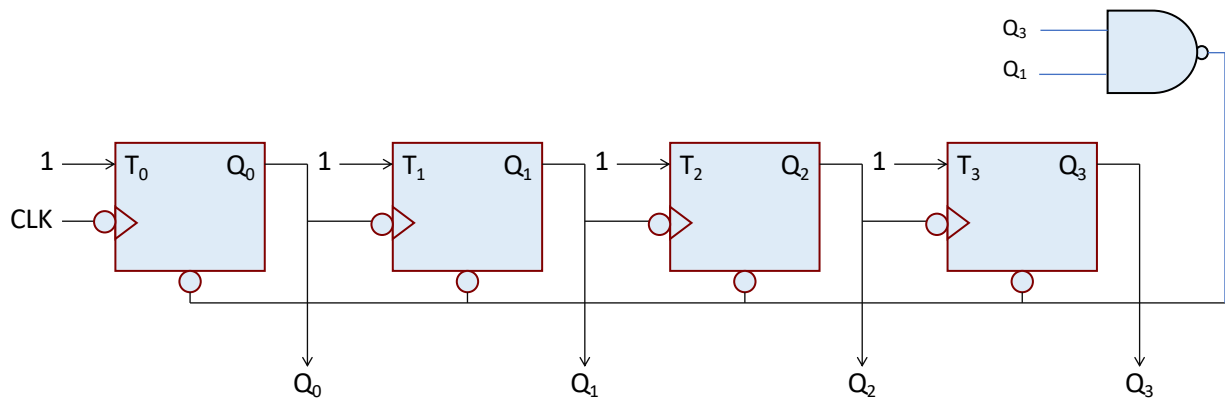


Switching Circuits and Logic Design (CS20202)

44

44

Example: Design of a decade (modulo-10) counter



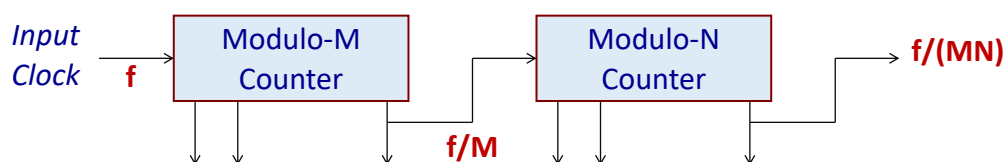
Switching Circuits and Logic Design (CS20202)

45

45

Cascading of Ripple Counters

- If a *modulo-M* and a *modulo-N* counter are connected in cascade, the combination will count *modulo-MN*.



Switching Circuits and Logic Design (CS20202)

46

46

Example: Design a modulo-1000 counter using decade counters

Switching Circuits and Logic Design (CS20202)

47

47

Example: Design a digital clock

Switching Circuits and Logic Design (CS20202)

48

48